

ECOLE NATIONALE DE LA STATISTIQUE ET DE L'ANALYSE DE
L'INFORMATION



PROJET DE TRAITEMENT DE DONNÉES

Élèves de première année

Rapport du projet de traitement de données

Étudiants :

Raphael Toreilles

Benjamin Blanc

Nathan Tiatia

Tuteur :

Clément Valot

Table des matières

Introduction	2
I - Présentation générale du projet	2
1 - Cahier des charges	2
Diagramme de classe	2
Diagramme d'activité	3
Diagramme d'architecture	5
2 - Gestion du projet par le groupe	5
Modèle de données	5
Le choix des Loaders	6
La visualisation	6
Les tests	7
II - La structure du code	7
Les Loaders	7
La programmation orientée objet	9
L'application/déroulement du programme	11
III - Les résultats	13
Conclusion	21
Annexe	22
Diagrammes des sous-menus	22

Introduction

Notre rapport présente une application concernant différents sports. Elle permet de modéliser et d'enrichir les données pour chaque joueur, équipe ou compétition, indifféramment du sport concerné. À la suite des succès des Jeux Olympiques et Paralympiques d'été à Paris ou d'hiver à Milan, une application telle que la notre satisfait la demande de nombreux individus souhaitant s'informer sur le monde du sport le plus efficacement possible.

Ce projet a nécessité la maîtrise d'une large base de données concernant les joueurs et joueuses, les matchs et les compétitions de chaque sport. Concrètement, nous nous sommes mis comme objectif de produire une application diffusant une interface à un utilisateur. Usant de cet interface, l'utilisateur a la possibilité de chercher un joueur, une joueuse, une équipe, un match ou une compétition parmi ceux présents dans la base de données initiales. L'intérêt premier de cette application est de visualiser les diverses statistiques sportives générales à tous les sports -classement, défaites, victoires, etc- et d'autres statistiques spécifiques au sport concerné allant des rebonds au basketball au nombre de dragons ou drakes tués par une équipe sur League of Legends. Ces statistiques se visualisent soit directement sur l'interface, soit à travers des graphiques que l'application s'occupe de mettre en oeuvre. Dans un second temps, l'application possède aussi la fonctionnalité de mettre à jour la base de données avec de nouveaux matchs et de mettre à jour automatiquement les statistiques après mise à jour. Nous avons fait le choix de garder 5 sports : Basketball, Football, Tennis, Volleyball et League of Legends.

Les données que nous avons sont d'abord sous le format de fichier CSV. Ce dernier, grâce à un module `parse_cv`, est converti en python sous forme d'une liste de dictionnaire. Le format de chaque dictionnaire est spécifique mais utilisable à l'aide des clés qu'un dictionnaire se doit de posséder. Chaque ligne du CSV est un dictionnaire avec l'en-tête comme clé. Cette disposition nous permet, comme nous le verrons ensuite, d'uniformiser des données qui sont différentes pour chaque fichier CSV et de traiter correctement la base de données. Enfin, afin de traiter ces données dans l'optique de l'application, nous avons choisi de modéliser chaque objet utilisé et modélisé au sein de l'application. Comme nous le verrons dans le diagramme de classes, nous avons choisi certains objets afin d'en faire une classe composés d'attributs et de méthodes afin de calculer les statistiques et de faire fonctionner l'interface de l'application.

I - Présentation générale du projet

1 - Cahier des charges

Le cahier des charges étant assez explicite, la majorité de cette partie se contentera de répondre aux exigences du projet par des diagrammes de différents types.

Diagramme de classe

Nous avons d'abord voulu respecter la demande de formation de classes et de leurs modélisations au sein du cahier des charges. En effet, "l'application permet de créer des objets modélisant au minimum des matchs, des compétitions, des équipes ou des joueurs et joueuses". Chacune de ces classes sont explicitement demandées et nous avons pris comme choix de former ces classes selon les méthodes et attributs suivants.

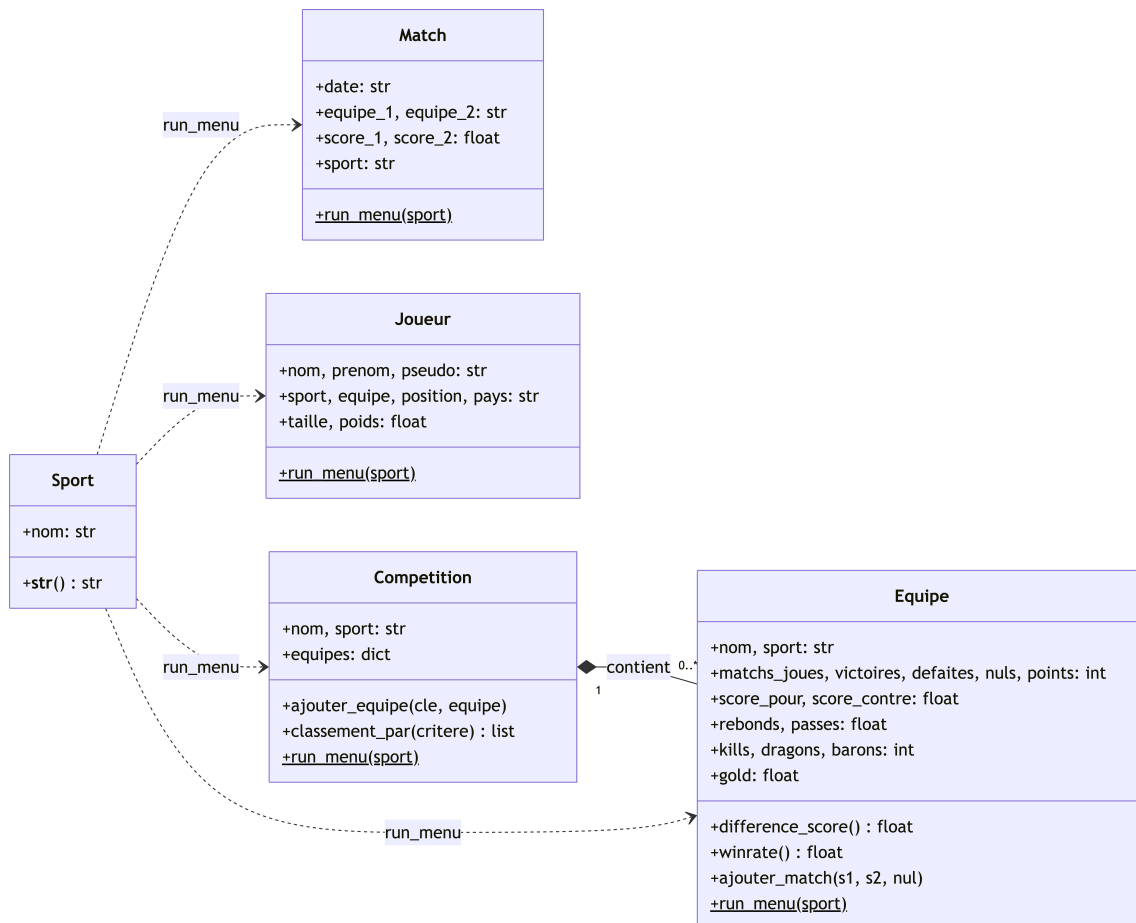


Diagramme d'activité

L'application doit être en interaction avec l'utilisateur. Ce dernier peut alors chercher des statistiques, mettre à jour les données et les traiter si nécessaire : "Il doit être possible d'ajouter les résultats d'une nouvelle compétition et de mettre à jour les caractéristiques des équipes ou des joueurs et joueuses en prenant en compte les nouveaux résultats." Le diagramme d'activité permet de mettre en exergue les relations entre l'application ou encore l'interface et l'utilisateur

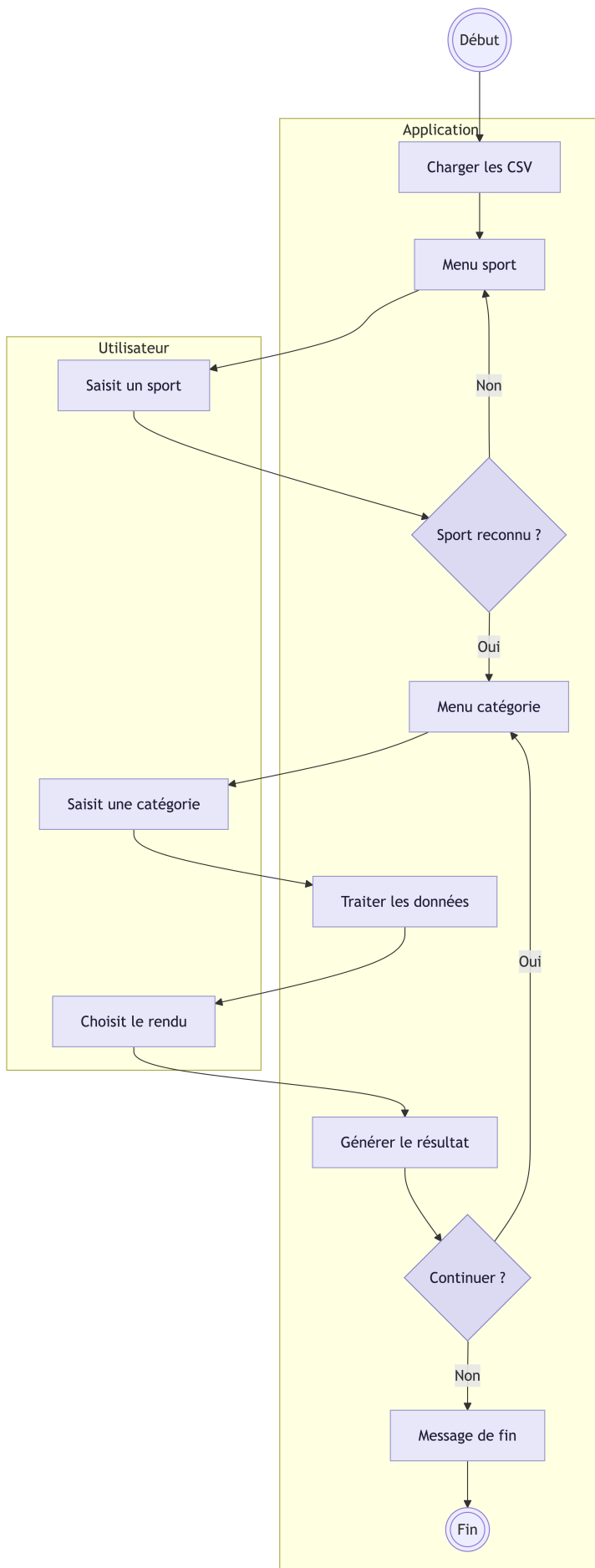
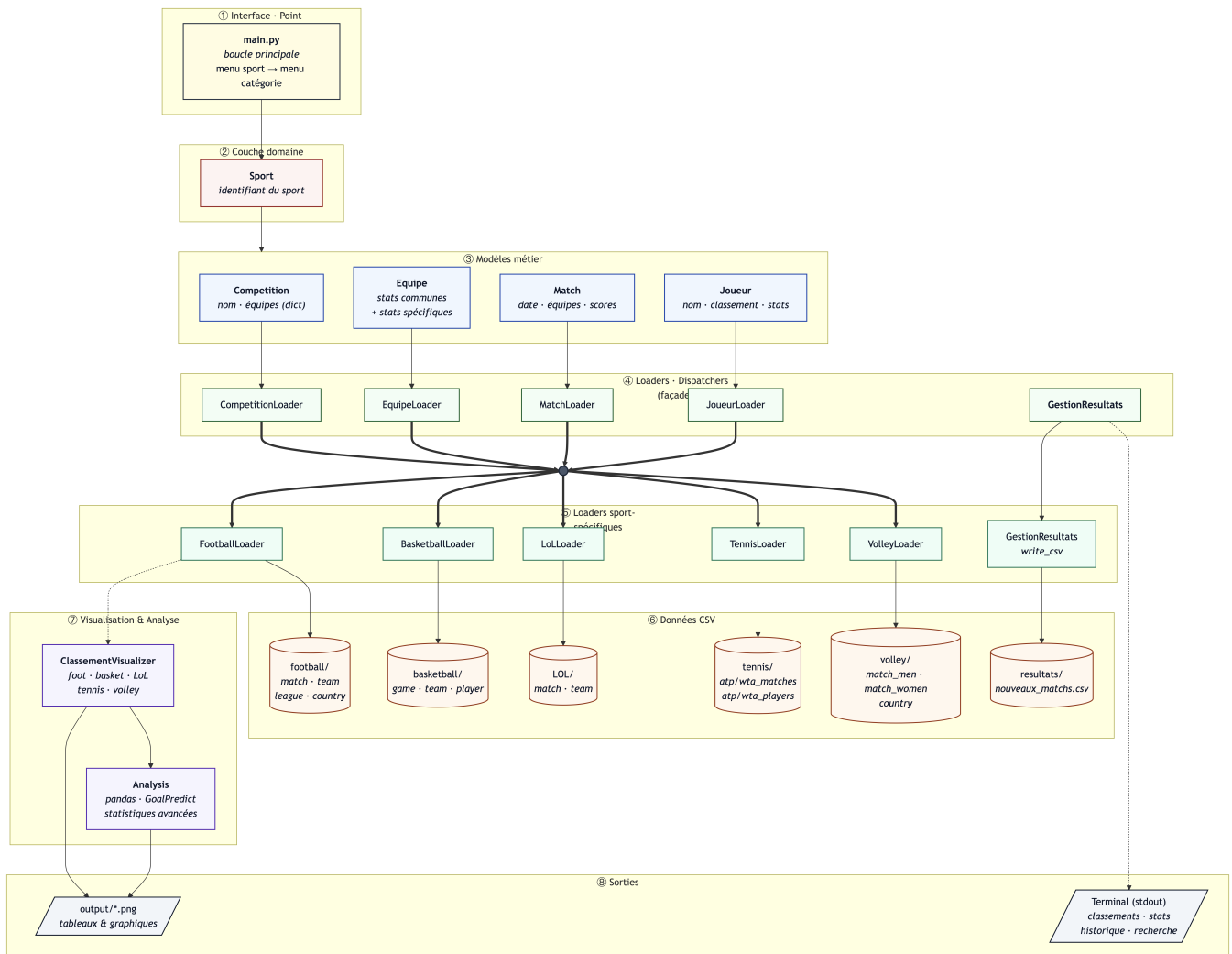


Diagramme d'architecture

Nous expliquerons en détail par la suite le fonctionnement de l'application. Le diagramme permet néanmoins de comprendre les différents niveaux sur lesquels l'application agit. Nous appellerons ces derniers des couches, soit des parties de l'architecture du code avec un but précis, ne pouvant modifier ou être modifié seulement par ses couches voisines. Cela nous permet de présenter une vue d'ensemble du projet avant de le détailler. Nous montrons ainsi l'organisation du code et les interactions entre modules.



2 - Gestion du projet par le groupe

Modèle de données

Pour notre application, nous avons choisi de traiter cinq sports : le football, le basketball, le tennis, le volleyball et l'e-sport (League of Legends). Les données initiales, qui représentent la matière première du projet, sont toutes hétérogènes d'un sport à l'autre, ce qui a pu constituer un premier obstacle à la mise en forme de notre application. Par exemple, 5 fichiers CSV viennent avec le dossier zip sur les data football : `country.csv`, `league.csv`, `match.csv`, `player.csv` et `team.csv`. En comparaison, le zip "tennis" possède les CSV suivant : `atp_matches_2024.csv`, `atp_players_2024.csv`, `wta_matches_2024.csv` et `wta_players_2024.csv`. Mais encore, l'hétérogénéité est présente au sein même des CSV. Pour le fichier `player.csv` de football, les caractéristiques des joueurs sont : `id`, `player_api_id`, `player_name`, `birthday`, `weight` (kg), `height` (cm). D'un autre côté, le fichier `wta_players_2024.csv` donne les caractéristiques suivantes pour les joueuses de tennis : `player_id`, `name_first`, `name_last`, `hand`, `dob`, `ioc`, `height`. Chaque

sport a son propre schéma de colonnes, ses propres identifiants, ses propres unités.

Football est la base de données la plus conséquente. Presque 26 000 lignes sur le fichier match.csv avec 33 colonnes sont répartis sur les 11 ligues européennes. Les 4 fichiers csv sont à joindre pour avoir des informations utiles : par exemple pour afficher “Ligue 1 — France — PSG 2-1 Lyon”, il faut traverser ces 4 fichiers. Le basketball a le dataset le plus enrichi avec 43 colonnes de stats pour les matchs d’une saison NBA : tirs à 2pts, tirs à 3pts, lancers francs (tentatives + réussis + pourcentage), rebonds offensifs et défensifs, passes décisives, interceptions, contres, balles perdues, fautes, points totaux. La base de données de League of Legends au contraire du football, est la plus petite. Chaque match contient : composition complète par rôle (top, jungle, mid, bot, support pour chaque équipe), les 5 picks et 5 bans de chaque côté, et les stats d’objectifs (dragons, barons, tourelles, gold, kills, assists, deaths). Le tennis est divisé en deux circuits avec des match ATP et WTA de 2024. Ce qui fait qu’on ne peut pas traiter de manière similaire les résultats dans ces deux tournois par exemple (l’un utilisant best of 3, le second best of 5). Néanmoins, ici aussi chaque statistiques de match est très détaillées : aces, doubles-fautes, points de service joués, 1er service entré, 1er et 2e service gagnés, jeux de service, balles de break sauvées/concédées et le temps du match en minutes. Enfin le dataset volleyball est le plus simplifié. Il concerne 26 matchs pour chaque sexe. La structure de match.csv est réduite à l’essentiel : date, phase de compétition (stage), codes pays des deux équipes, et nombre de sets gagnés par chacun. En accord avec le cahier des charges, nous avons décidé d’accepter ces différences au sein de chaque CSV. C’est surtout à travers les Loaders que nous avons cherché à réduire cette hétérogénéité.

Le choix des Loaders

Les 5 sports ont des fichiers CSV incompatibles entre eux. Il est nécessaire de construire des Loaders afin d’éviter une redondance dans le code de l’application avec, à chaque fonctionnalité, un `if sport == “football” ... elif sport == “basketball”` etc. Nous expliquerons en détail l’utilisation des Loaders et leur utilité dans la seconde partie. Nous avons choisi de former 4 classes de Loaders. Une correspondant à chaque classe du diagramme : MatchLoader, JoueurLoader, CompetitionLoader, EquipeLoader. MatchLoader normalise les données hétérogènes en objets Match uniformes pour tous les sports. JoueurLoader charge la liste des joueurs et affiche les statistiques pour chaque sports. EquipeLoader affiche les statistiques d’une équipe précise au sein d’une saison. Enfin CompetitionLoader affiche les classements d’une compétition sous plusieurs angles (points, attaque, défense, rebonds...). Dans cette optique nous avons choisis d’utiliser pandas afin de construire des dataframes normalisés avec les Loader, mais aussi de filter certaines données et statistiques, voire les visualiser. Nous pouvons aussi identifier le cas complexe de la classe GestionResultat. Ce dernier s’occupe d’ajouter et de mettre à jour les données imputés via l’interface. Les nouveaux matchs sont alors concaténés à la liste de matchs déjà existant. Le problème étant alors la mise à jour des équipes et des compétitions en lien avec ces nouveaux matchs, GestionResultat s’occupe d’appliquer la mise à jour à la classe Competition et Match grâce aux méthodes `appliquer_a_equipe()` et `appliquer_a_competition()`. Les tests ont permis la bonne utilisation de GestionResultat ainsi que de tous les Loaders, nous approfondirons quelque peu par la suite ces tests.

La visualisation

Afin de permettre la visualisation nous avons nécessité l’utilisation de pandas et de matplotlib. Si pandas servaient d’abord à normaliser les données, il est par la suite devenue un outil de modélisation et de visualisation statistique de par l’utilisation de ses dataframes. Pour ce faire, ce sont des calculs entre les dataframes qui permettent par la suite de mettre en place des graphiques. Par exemple : `pd.concat([df[“pts_home”], df[“pts_away”]])` fusionne les points domicile et extérieur en une seule série pour calculer la distribution globale des scores. Ensuite nous modélisons avec matplotlib, une bibliothèque de visualisation Python. Ce dernier permet de faire les graphiques statistiques : `ax.hist` pour les distributions ou encore `ax.pie` pour les camemberts par exemple. Ces graphiques sont générées

sous forme de PNG à l'aide de `matplotlib.use("Agg")`. Afin d'éviter la répétition du code pour chaque graphique de formes similaires, on utilise la fonction `_style(ax, titre="", xlabel="", ylabel="")` qui permet de mettre en place les réglages nécessaires pour éviter la répétition. La visualisation se fait avec le module `GraphiqueSport` qui génère des graphiques couvrant tout les sports. Ces derniers sont générés dans la section output.

Les tests

Dans l'ensemble, 71 tests ont été codés. Les 71 passent tous en 0.6 secondes. Nous avons décidé de tester en majorité les classes demandées explicitement par le cahier des charges. Le module `test_Equipe.py` est le fichier le plus dense (15 tests). Il vérifie toute la logique de `ajouter_match` : victoire, défaite, nul, cumul sur plusieurs matchs, `nul_possible=False` pour les sports sans match nul, et les propriétés calculées `difference_score` et `winrate`. Par la suite, `test_Compétition.py` vérifie que `classement_par` produit un classement trié correctement selon n'importe quel attribut d'`Equipe`, et qu'une équipe existante peut être remplacée. `test_Match.py` vérifie que les champs optionnels spécifiques à chaque sport (par exemple `patch` pour LoL ou `surface` pour le tennis) sont bien `None` par défaut et correctement assignés quand ces derniers sont fournis. De par la complexité de la classe `GestionResultat`, `test_GestionResultat` est le plus fourni en tests. Le module `test_registries.py` vérifie que les 5 sports sont bien enregistrés dans les 4 Loaders après l'importation des bases de données initiales :

```
def test_match_loaders_tous_enregistres():
    assert set(MatchLoader._loaders.keys()) == {"basketball", "football", "LOL", "tennis", "volley"}
```

 Néanmoins, beaucoup des fonctionnalités ne sont pas couvertes. Les Loaders et les ont très peu de tests et les Visualizers n'en ont aucun. La difficulté réside d'une part ici dans la lecture de fichier CSV. D'autre part la plupart des fonctionnalités nécessite un `input()`. La stratégie que nous avons adoptée est donc de concentrer les tests sur les classes de base et `GestionResultat`.

II - La structure du code

Les Loaders

Tout d'abord, afin d'importer correctement les données, nous avons choisi l'utilisation de loaders. Les fichiers bruts étant au format CSV, nous avons utilisé la librairie `pandas` afin de transformer ces fichiers en `DataFrames`, c'est-à-dire en tableaux de données facilement manipulables en Python. Cette première étape permet de lire les fichiers contenant les informations sur les matchs, les joueurs, les équipes ou encore les compétitions. Le but des Loaders est de servir d'intermédiaire entre les données et le reste du programme. Les fichiers CSV contiennent les données utilisées durant le projet, mais elles ne peuvent pas être utilisées directement, car leur structure varie selon les sports et les fichiers. Les loaders ont donc pour rôle de lire ces fichiers, de sélectionner les informations utiles, de nettoyer et transformer les données, puis de convertir les lignes en objets Python utilisables par l'application. Ils permettent ainsi de mettre dans le même format toutes les données issues des fichiers csv afin que le reste du programme puisse fonctionner de manière identique. Cette approche améliore l'organisation du code et permet d'ajouter plus facilement de nouvelles sources de données.

Les fichiers CSV contiennent donc les données, mais la structure n'est pas la même d'un fichier à l'autre. Par exemple, pour les joueurs, certains fichiers utilisent des identifiants (ID) alors que d'autres non, et chaque sport possède également ses propres statistiques spécifiques. Il en est de même pour les matchs, les équipes ou les compétitions, dont les colonnes et l'organisation varient selon les sources de données. Afin de gérer cette hétérogénéité, les loaders utilisent plusieurs méthodes de la librairie `pandas`. Certaines colonnes sont sélectionnées afin de ne conserver que les informations utiles au programme. Des transformations sont également appliquées pour convertir certains identifiants en chaînes de caractères. Enfin, un nouveau `DataFrame` standardisé est reconstruit. Pour cela, une ligne est transformée en dictionnaire, et avec la méthode `(**r)`, chaque ligne est transformée en classe avec comme paramètre les valeurs du dictionnaire. Les loaders du projet sont séparés en deux catégories : les loaders génériques et

les loaders spécifiques. Les loaders génériques, comme MatchLoader, JoueurLoader, EquipeLoader ou CompetitionLoader, ne lisent pas directement les fichiers CSV. Leur rôle est de servir d'intermédiaires entre le programme et les loaders spécialisés. Ils permettent au reste de l'application d'utiliser toujours les mêmes méthodes, quel que soit le sport choisi.

```
@staticmethod
def register(sport_nom: str, loader) -> None:
    JoueurLoader._loaders[sport_nom] = loader

def load_all_joueurs(self, sport: Sport) -> list[Joueur]:
    """Charge tous les joueurs du sport sélectionné.

    Parameters
    -----
    sport : Sport
        Sport dont on veut charger les joueurs.

    Returns
    -----
    list[Joueur]
        Liste de joueurs.
    """
    loader = self._loaders.get(sport.nom)
```

FIGURE 1 : Loader

Les loaders spécifiques, comme FootballMatchLoader, BasketballJoueurLoader ou TennisCompetitionLoader, sont quant à eux responsables du traitement réel des données. Ils lisent les fichiers CSV associés à leur sport, sélectionnent les colonnes utiles, nettoient et transforment les données, puis créent les objets Python utilisés dans le programme. Cette séparation permet d'avoir une architecture plus claire et plus modulaire, où chaque loader possède une responsabilité précise. Afin de relier les loaders génériques aux loaders spécifiques, le projet utilise un système de registre avec des instructions comme MatchLoader.register("football", FootballMatchLoader). Lors de l'import des fichiers dans **main.py**, ces lignes sont automatiquement exécutées et enregistrent dans un dictionnaire quel loader doit être utilisé pour chaque sport. Ainsi, lorsque l'utilisateur choisit un sport dans l'application, le loader générique peut retrouver automatiquement le loader spécialisé correspondant et l'utiliser pour charger les données. Ce système permet d'ajouter facilement un nouveau sport sans modifier le reste du programme : il suffit de créer un nouveau loader spécifique puis de l'enregistrer avec register().

```

class BasketballJoueurLoader:
    DATA_PLAYERS = "data/basketball/player.csv"
    DATA_TEAMS = "data/basketball/team.csv"

    def load_all_joueurs(self) -> list[Joueur]:
        df_players = pd.read_csv(self.DATA_PLAYERS)
        df_teams = pd.read_csv(self.DATA_TEAMS)
        teams = {}
        for _, row in df_teams.iterrows():
            teams[str(row["id"])] = row["full_name"]
        joueurs = []
        for _, row in df_players.iterrows():
            joueurs.append(
                Joueur(
                    nom=str(row["last_name"]),
                    prenom=str(row["first_name"]),
                    sport="basketball",
                    equipe=teams.get(str(row["team_id"]), str(row["team_id"])),
                    position=str(row["position"])
                    if pd.notna(row["position"])
                    else None,
                    date_naissance=str(row["birthdate"])
                    if pd.notna(row["birthdate"])

```

FIGURE 2 : Loader

La programmation orientée objet

Nous avons utilisé la programmation orientée objet dans ce projet afin de représenter de manière claire les différentes entités manipulées par l'application, comme les matchs, les joueurs, les équipes ou les compétitions. Le projet manipulant de nombreuses données provenant de sports différents, cela permet d'organiser le code en séparant les responsabilités de chaque classe. Chaque objet possède ainsi ses propres attributs et méthodes, ce qui facilite la gestion des données, les traitements statistiques et les visualisations. Cela permet aussi d'ajouter plus facilement de nouvelles données.

Classe Joueur La classe Joueur représente un joueur appartenant à un sport donné. Un joueur est défini par son nom et son sport, mais la classe contient d'autres paramètres optionnels qui correspondent aux informations des joueurs, comme la taille ou le poste, qui ne seront présents que si l'information est dans les fichiers d'import. (Par exemple, la taille n'est pas présente pour les joueurs de LOL car cette information n'est pas pertinente). Afin de simplifier, nous avons créé un seul attribut position qui peut fonctionner pour plusieurs

sports (basket, LOL ...), bien que ces positions soient spécifiques à chaque sport

La classe possède aussi des méthodes permettant de trier les joueurs selon certaines caractéristiques afin d'avoir des statistiques par la suite (exemple : `__chercher__par__nom`, `__chercher__par__pays`)

```

class Joueur:
    """Représente un joueur sportif, tous sports confondus.

    Parameters
    -----
    nom : str
        | Nom de famille ou nom complet du joueur.
    sport : str
        | Nom du sport pratiqué.
    prenom : str, optional
        | Prénom du joueur.
    pseudo : str, optional
        | Pseudo en jeu, spécifique à l'e-sport.
    equipe : str, optional
        | Nom de l'équipe ou du circuit (ex : "ATP").
    position : str, optional
        | Poste ou rôle (ex : "Forward", "jungle").
    date_naissance : str, optional
        | Date de naissance au format AAAA-MM-JJ.
    taille : float, optional
        | Taille en centimètres.
    poids : float, optional
        | Poids en kilogrammes.
    pays : str, optional
        | Pays ou code pays du joueur.
    main : str, optional
        | Main dominante, spécifique au tennis (ex : "R", "L").
    """

```

FIGURE 3 : joueur

Classe Équipe

La classe Equipe représente une équipe. Elle est définie par un nom, un sport et une abréviation. Des statistiques sont aussi présentes en attribut, comme le nombre de matchs joués, le nombre de matchs gagnés ... Ces statistiques sont obtenues en agréant depuis le fichier des matchs par équipe. Il existe aussi des statistiques présentes seulement pour un sport, on peut prendre comme exemple les dragons pour LOL. Nous avons fait le choix de mettre les statistiques dans la classe équipe plutôt que dans la classe match car tout d'abord, comme nous le verrons après, la classe match est très simplifiée, et aussi car nous avons pensé qu'il était plus intéressant pour un utilisateur d'avoir des statistiques sur l'année plutôt que par match. En effet, de cette manière il est plus facile de comparer entre les équipes, et cela représente mieux la réussite d'une équipe. Classe Match La classe Match représente une rencontre sportive entre deux équipes ou deux joueurs appartenant à un sport donné. Un match est défini principalement par une date, deux participants (equipe_1 et equipe_2), leurs scores respectifs ainsi que le sport concerné. La classe contient également certains paramètres optionnels correspondant à des informations spécifiques à certains sports, comme la surface et le tournoi pour le tennis, ou encore le patch pour League of Legends. Ces attributs ne sont présents que si les informations existent dans les fichiers d'import. Afin d'uniformiser la structure des données entre les sports, nous avons choisi d'utiliser des attributs génériques comme score_1 et score_2, même si la signification exacte du score peut varier selon le sport (buts au football, points au basketball, kills pour LOL, sets au volley...).

La classe Match est volontairement plus simple que la classe Equipe, car nous avons fait le choix de stocker la majorité des statistiques agrégées directement dans les équipes plutôt que dans chaque match individuellement. Les objets Match servent donc principalement à représenter les données brutes des rencontres et à alimenter ensuite les statistiques des équipes et des compétitions. Cette simplification permet également d’avoir une structure commune plus facilement adaptable à des sports très différents. Tout comme la classe Joueur, elle possède aussi des méthodes utilisées par l’application afin d’obtenir des statistiques et des rendus visuels.

Classe Competition La classe Competition représente une compétition ou un championnat regroupant plusieurs équipes appartenant à un sport donné. Une compétition est définie principalement par son nom et son sport. Elle contient une méthode lui permettant de rajouter des équipes à la compétition et de les stocker dans un dictionnaire afin de pouvoir les retrouver et les manipuler facilement par la suite. La classe possède également des méthodes permettant de trier les équipes selon différents critères statistiques, comme le nombre de points, le nombre de victoires ou la différence de score, afin de générer des classements. Cette classe joue donc un rôle central dans l’organisation des données sportives, puisqu’elle permet de regrouper les équipes d’un même championnat et de comparer leurs performances sur l’ensemble d’une saison.

L’application/déroulement du programme

Afin de lancer l’application, l’utilisateur passe par le terminal, il doit exécuter le fichier **main.py**. Ce fichier joue le rôle de point d’entrée du programme. Sa première fonction est d’importer l’ensemble des loaders afin d’utiliser les données souhaitées par l’utilisateur. Le fichier main contient donc des fonctions permettant la visualisation de l’appli, comme l’en-tête. Ensuite, une fonction demande à l’utilisateur le sport qu’il souhaite consulter, puis la catégorie des données qu’il souhaite consulter.

```
— Matches —
1 Chercher les matchs d'une équipe
2 Chercher les matchs entre deux équipes
3 Chercher les matchs à une date précise
4 Chercher les matchs où le score dépasse un seuil
5 Statistiques agrégées
0 Retour
```

FIGURE 4 : choix

Ici, si l’utilisateur choisit joueur, équipe, compétition ou match, l’appli fera appel à la méthode `run_menu` qui est présente dans chacune de ces classes, et qui fera apparaître un menu déroulant faisant appel aux méthodes de cette même classe, afin d’obtenir des statistiques. Voici un exemple avec la classe match :


```

@staticmethod
def run_menu(sport: Sport) -> None:
    """Affiche le menu interactif des matchs pour un sport donné.

    Charge tous les matchs une seule fois, puis boucle sur les requêtes
    jusqu'à ce que l'utilisateur choisisse de revenir.

    Parameters
    -----
    sport : Sport
    |     Sport sélectionné par l'utilisateur.
    """
    from src.loader.MatchLoader import MatchLoader

    matchs = MatchLoader().load_all_matches(sport)
    print(f" {len(matchs)} matchs chargés pour {sport.nom}.")

    while True:
        print("\n — Matchs —————")
        print(" 1 Chercher les matchs d'une équipe")
        print(" 2 Chercher les matchs entre deux équipes")
        print(" 3 Chercher les matchs à une date précise")
        print(" 4 Chercher les matchs où le score dépasse un seuil")
        print(" 5 Statistiques agrégées")
        print(" 0 Retour")

        choix = input("\n Votre choix : ").strip()

        if choix == "0" or choix.lower() in ("retour", "q"):
            break
        elif choix == "1":
            Match._matchs_equipe(matchs)
        elif choix == "2":
            Match._matchs_entre_deux_equipes(matchs)
        elif choix == "3":
            Match._matchs_par_date(matchs)

```

FIGURE 5 : Run_menu

Ici, si l'utilisateur sélectionne dans 'match' tous les matchs d'une certaine équipe, au sein de la classe équipe run_menu fera appel à la méthode suivante, et les résultats s'afficheront dans le terminal :

Pour les graphiques, on fait appel au fichier GraphiqueSport, qui contient des fonctions permettant de construire des graphiques à l'aide des bibliothèques pandas, numpy ainsi que matplotlib. Il existe des graphiques spécifiques pour chaque sport afin de mettre en avant des statistiques adaptées aux données disponibles. Par exemple, pour le football, certains graphiques représentent le nombre de buts par journée ou la comparaison entre attaque et défense des équipes, là ou pour le basketball, des visualisations sont réalisées sur la distribution des scores ou les joueurs réalisant le plus de rebonds. Il existe aussi une fonction permettant à l'utilisateur d'ajouter lui-même des résultats de match. Cette fonctionnalité passe par la classe GestionResultats, qui permet de sauvegarder de nouveaux matchs

saisis directement depuis le terminal. L'utilisateur renseigne les informations nécessaires, comme le sport, la date, les équipes concernées ainsi que les scores du match. Ces nouvelles données sont ensuite enregistrées dans un fichier CSV situé dans data/resultats/nouveaux_matches.csv. Ces nouveaux matchs sont donc par la suite utilisés lors des calculs des statistiques.

III - Les résultats

Etude 1 :

L'utilisateur est l'entraîneur du Paris Saint-Germain, il souhaite préparer son match de vendredi face à l'Olympique de Marseille. Il effectue alors des recherches statistiques sur l'équipe.

```

Base de données sportive – Résultats & Statistiques

Sports : basketball | football | LOL | tennis | volley
(q pour quitter)

Quel sport ? football

— FOOTBALL —
match      Résultats et statistiques de matchs
joueur     Recherche et statistiques des joueurs
equipe     Statistiques d'une équipe
competition Classements de la compétition
graphiques Graphiques et classements visuels (PNG)
ajouter    Enregistrer le résultat d'un match
historique Consulter les résultats enregistrés

(retour = changer de sport | q = quitter)

Quelle catégorie ? equipe

Nom de l'équipe : Olympique de Marseille

```

FIGURE 6 : Premières actions, choisir la catégorie équipe et le nom de celle-ci

```

Saisons : 2008/2009, 2009/2010, 2010/2011, 2011/2012, 2012/2013, 2013/2014, 2014/2015, 2015/201
6
Saison (ex: 2014/2015) : 2015/2016

=== Olympique de Marseille – 2015/2016 ===

Matches joués   : 38
Victoires       : 10  Nuls : 18  Défaites : 10
Buts marqués    : 48  (moy: 1.26/match)
Buts encaissés  : 42  (moy: 1.11/match)
Différence      : +6  | Points : 48  | Winrate : 26.3%

```

FIGURE 7 : Seconde action, choisir la période + Résultat

Il décide ensuite de comparer les deux équipes.

```

— FOOTBALL —
match      Résultats et statistiques de matchs
joueur     Recherche et statistiques des joueurs
equipe     Statistiques d'une équipe
compétition Classements de la compétition
graphiques Graphiques et classements visuels (PNG)
ajouter    Enregistrer le résultat d'un match
historique Consulter les résultats enregistrés

(retour = changer de sport | q = quitter)

Quelle catégorie ? graphiques

— Graphiques – FOOTBALL —
1 Buts moyens par journée
2 Distribution des scores
3 Attaque vs Défense (scatter bulle)
4 Comparaison deux équipes
0 Retour

Votre choix : 4
Équipe 1 : Paris Saint-Germain
Équipe 2 : Olympique de Marseille

Graphique généré : output/comparaison_football_Paris_Sain_Olympique_.png

```

FIGURE 8 : Première action, choix de la catégorie graphique + choix des deux équipes

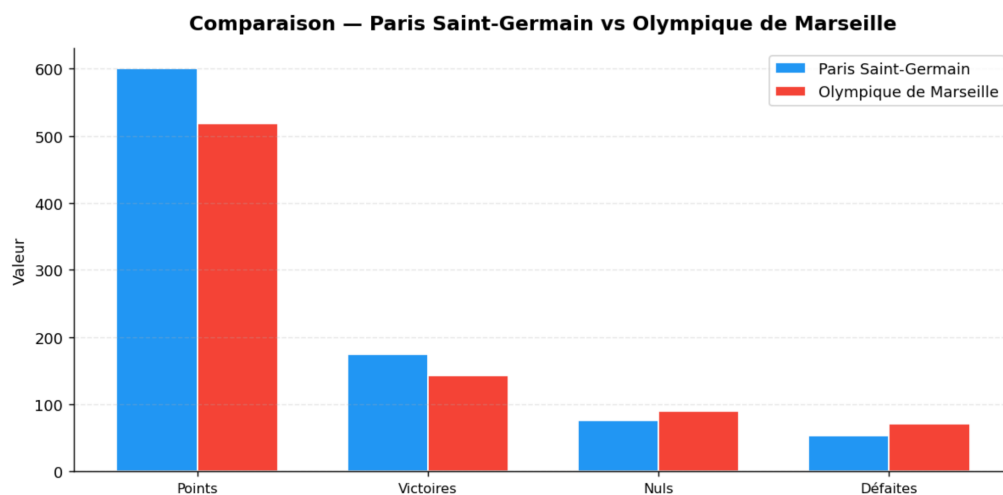


FIGURE 9 : Résultat

Etude 2 :

L'utilisateur, un montpellierain, est nostalgique de la saison 2011/2012.

Base de données sportive – Résultats & Statistiques

Sports : basketball | football | LOL | tennis | volley
(q pour quitter)

Quel sport ? football

FOOTBALL

match	Résultats et statistiques de matchs
joueur	Recherche et statistiques des joueurs
equipe	Statistiques d'une équipe
competition	Classements de la compétition
graphiques	Graphiques et classements visuels (PNG)
ajouter	Enregistrer le résultat d'un match
historique	Consulter les résultats enregistrés

(retour = changer de sport | q = quitter)

Quelle catégorie ? competition

FIGURE 10 : Première action, choix de la catégorie compétition

Que voulez-vous faire ?

- 1 - Classement d'une ligue pour une saison
- 2 - Meilleures attaques d'une ligue
- 3 - Meilleures défenses d'une ligue
- 4 - Classement général d'un championnat (toutes saisons)

Votre choix : 1

Ligues disponibles :

- 1 - Belgium Jupiler League (Belgium)
- 1729 - England Premier League (England)
- 4769 - France Ligue 1 (France)
- 7809 - Germany 1. Bundesliga (Germany)
- 10257 - Italy Serie A (Italy)
- 13274 - Netherlands Eredivisie (Netherlands)
- 15722 - Poland Ekstraklasa (Poland)
- 17642 - Portugal Liga ZON Sagres (Portugal)
- 19694 - Scotland Premier League (Scotland)
- 21518 - Spain LIGA BBVA (Spain)
- 24558 - Switzerland Super League (Switzerland)

ID de la ligue (0 = toutes) : 4769

Saisons : 2008/2009, 2009/2010, 2010/2011, 2011/2012, 2012/2013, 2013/2014, 2014/2015, 2015/2016

Saison (ex: 2014/2015) : 2011/2012

FIGURE 11 : Deuxième action, choix de la ligue et de l'année

=== France Ligue 1 – 2011/2012 ===									
#	Équipe	MJ	V	N	D	BP	BC	DB	Pts
1	Montpellier Hérault SC	38	25	7	6	68	34	+34	82
2	Paris Saint-Germain	38	23	10	5	75	41	+34	79
3	LOSC Lille	38	21	11	6	72	39	+33	74
4	Olympique Lyonnais	38	19	7	12	64	51	+13	64
5	Girondins de Bordeaux	38	16	13	9	53	41	+12	61
6	Stade Rennais FC	38	17	9	12	53	44	+9	60
7	AS Saint-Étienne	38	16	9	13	49	45	+4	57
8	Toulouse FC	38	15	11	12	37	34	+3	56
9	Évian Thonon Gaillard FC	38	13	11	14	54	55	-1	50
10	Olympique de Marseille	38	12	12	14	45	41	+4	48
11	AS Nancy-Lorraine	38	11	12	15	38	48	-10	45
12	Valenciennes FC	38	12	7	19	40	50	-10	43
13	FC Sochaux-Montbéliard	38	11	9	18	40	60	-20	42
14	OGC Nice	38	10	12	16	39	46	-7	42
15	AC Ajaccio	38	9	14	15	40	61	-21	41
16	Stade Brestois 29	38	8	17	13	31	38	-7	41
17	FC Lorient	38	9	12	17	35	49	-14	39
18	SM Caen	38	9	11	18	39	59	-20	38
19	Dijon FCO	38	9	9	20	38	63	-25	36
20	AJ Auxerre	38	7	13	18	46	57	-11	34

FIGURE 12 : Résultat

Etude 3 :

Un fan de basket entend parler d'un Jonathan Isaac. Mais il ne le connaît pas, il décide alors de lancer une requête pour avoir des informations sur lui.

Base de données sportive – Résultats & Statistiques	
Sports : basketball football LOL tennis volley (q pour quitter)	
Quel sport ? basketball	
— BASKETBALL —	
match	Résultats et statistiques de matchs
joueur	Recherche et statistiques des joueurs
equipe	Statistiques d'une équipe
competition	Classements de la compétition
graphiques	Graphiques et classements visuels (PNG)
ajouter	Enregistrer le résultat d'un match
historique	Consulter les résultats enregistrés
(retour = changer de sport q = quitter)	
Quelle catégorie ? joueur	
431 joueurs chargés pour basketball.	

FIGURE 13 : Première action, choix de la catégorie

— Joueurs —

- 1 Chercher un joueur par nom
- 2 Chercher les joueurs d'une équipe
- 3 Chercher les joueurs par position
- 4 Statistiques des joueurs
- 0 Retour

Votre choix : 1

Nom du joueur : Jonathan

1 résultat(s) :

Joueur	: Jonathan Isaac
Poste	: Forward
Équipe actuelle	: Orlando Magic
Naissance	: 1997-10-03 (28 ans)
Taille	: 208 cm
Poids	: 104.3 kg
Numéro maillot	: #1

FIGURE 14 : Seconde action, cherche le joueur précis + résultat

Etude 4 :

Deux amis débattent sur la meilleure équipe entre les Lakers et les Celtics. Ils décident alors de regarder les deux derniers matchs les opposant.

```
Base de données sportive – Résultats & Statistiques

Sports : basketball | football | LOL | tennis | volley
(q pour quitter)

Quel sport ? basketball

— BASKETBALL —
match      Résultats et statistiques de matchs
joueur     Recherche et statistiques des joueurs
equipe     Statistiques d'une équipe
competition Classements de la compétition
graphiques Graphiques et classements visuels (PNG)
ajouter    Enregistrer le résultat d'un match
historique Consulter les résultats enregistrés

(retour = changer de sport | q = quitter)

Quelle catégorie ? match

1314 matchs chargés pour basketball.
```

FIGURE 15 : Première action, choix de la catégorie

```
— Matches —
1 Chercher les matchs d'une équipe
2 Chercher les matchs entre deux équipes
3 Chercher les matchs à une date précise
4 Chercher les matchs où le score dépasse un seuil
5 Statistiques agrégées
0 Retour

Votre choix : 2
Équipe 1 : Los Angeles Lakers
Équipe 2 : Boston Celtics

2 résultat(s) :

[basketball] 2022-12-13 | Los Angeles Lakers 118 - 122 Boston Celtics
[basketball] 2023-01-28 | Boston Celtics 125 - 121 Los Angeles Lakers
```

FIGURE 16 : Deuxième action, choix des équipes + résultat

Etude 5 :

Une étude est menée par l'INSEE par rapport à la domination de certaines nationalités au tennis. Les chercheurs souhaitent connaître les nationalités des joueurs pro de tennis.


```
=====
Base de données sportive – Résultats & Statistiques
=====

Sports : basketball | football | LOL | tennis | volley
(q pour quitter)

Quel sport ? tennis

— TENNIS —
match      Résultats et statistiques de matchs
joueur     Recherche et statistiques des joueurs
equipe     Statistiques d'une équipe
competition Classements de la compétition
graphiques Graphiques et classements visuels (PNG)
ajouter    Enregistrer le résultat d'un match
historique Consulter les résultats enregistrés

(retour = changer de sport | q = quitter)

Quelle catégorie ? graphiques
```

FIGURE 17 : Première action, choix de la catégorie

```
— Graphiques – TENNIS —
1 Top 10 joueurs ATP
2 Top 10 joueuses WTA
3 Surface ATP (camembert)
4 Surface WTA (camembert)
5 Taille ATP vs WTA
6 Nationalités ATP (camembert)
7 Nationalités WTA (camembert)
0 Retour

Votre choix : 6

Graphique généré : output/tennis_nationalite_atp.png
```

FIGURE 18 : Seconde action, choix du graphique

Répartition par nationalité — ATP 2024

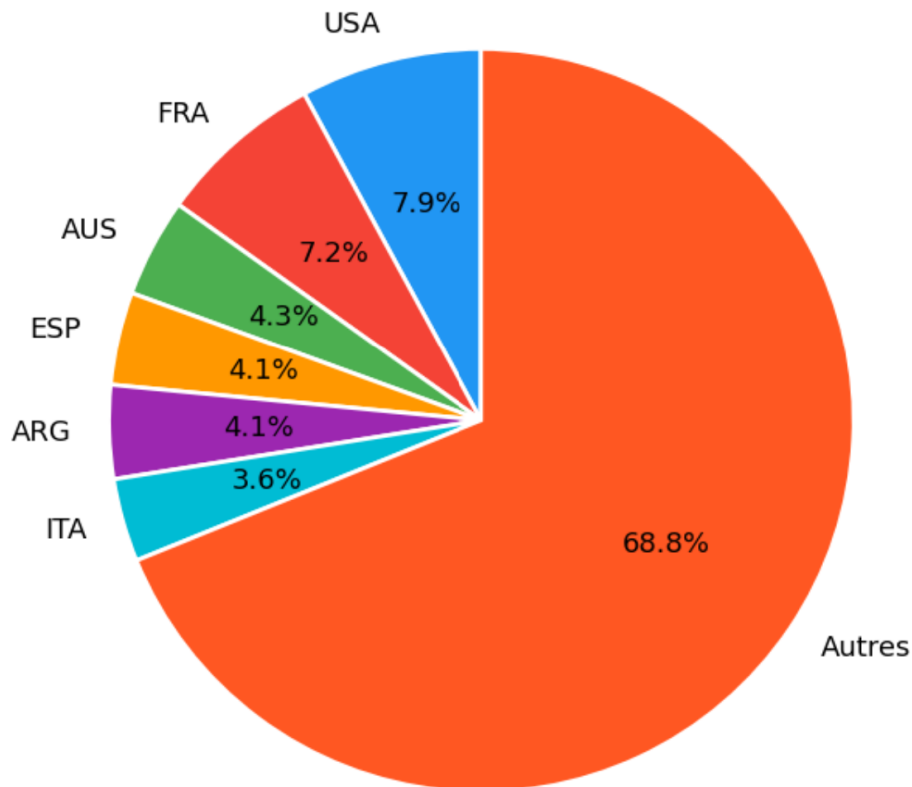


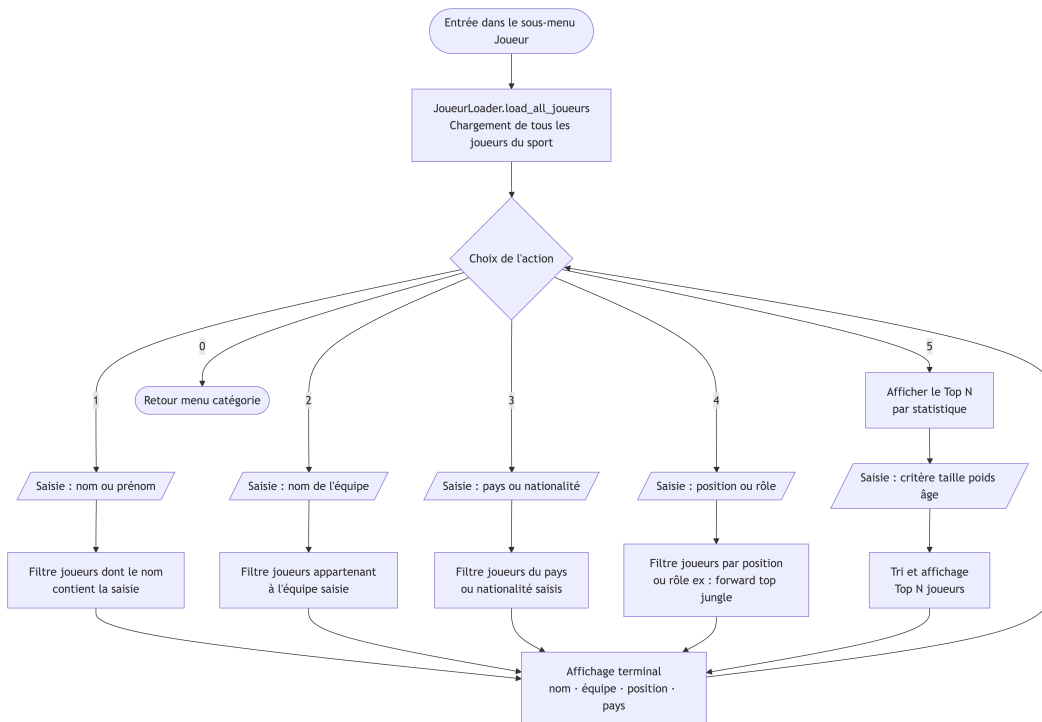
FIGURE 19 : Résultat 1

Conclusion

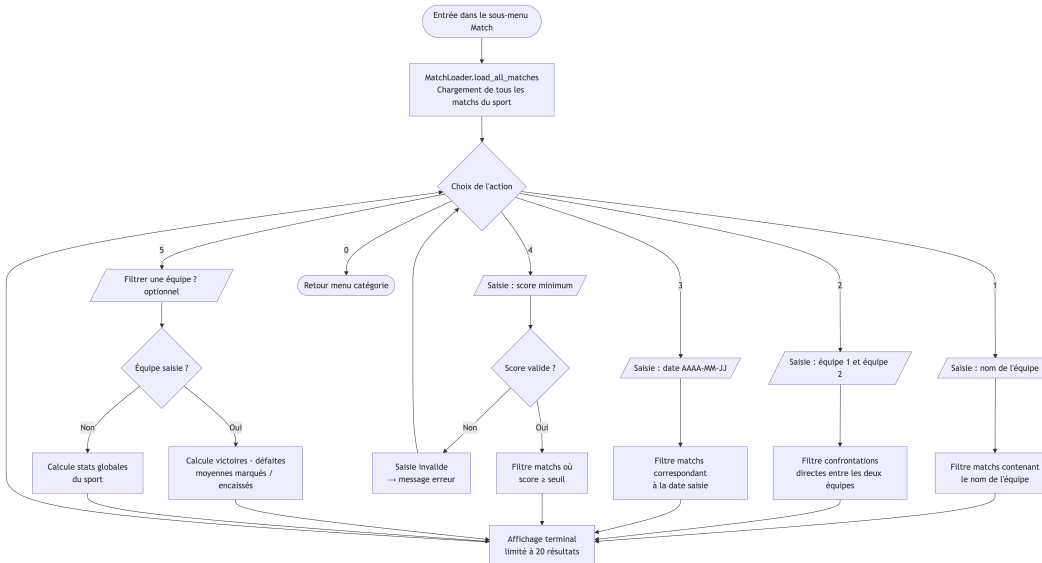
Concernant les données que nous avons utilisées lors de ce projet, nous nous sommes concentrés sur les premiers jeux de données qui nous ont été fournis. Cela nous a permis de plus nous focaliser sur la partie application et rendus pour l'utilisateur. Nous avons ainsi pu consacrer davantage de temps à la conception du programme, à la mise en place des loaders, à l'organisation des classes et à la génération des statistiques et visualisations. Malgré cela, les fichiers utilisés restaient déjà très variés et suffisamment riches pour mettre en œuvre des traitements, des classements et des graphiques adaptés à chaque sport. De plus, nous avons tout de même essayé de faire en sorte de garder une architecture de code facilitant la mise en place de nouveaux jeux de données, hormis sur la partie graphique, où nous nous sommes plus concentrés sur le fait de fournir des graphiques pertinents pour chaque sport. Ce projet a été pour nous une opportunité de nous familiariser avec la programmation et plus particulièrement celle orientée objet. Bien que le sujet ait été flou au départ, cela nous a entraînés à comprendre ce qui nous était demandé, et comment le mettre en place à l'aide de classes et de méthodes. Nous avons aussi appris la programmation en Python directement liée à la data science, en utilisant des bibliothèques comme pandas ou matplotlib. Cela a aussi été pour nous une opportunité de voir et comprendre comment se déroulent des projets à plus grande échelle, et nous a appris des choses que nous ne savions pas forcément bien faire, comme la délégation et la répartition des tâches, ou le travail à plusieurs sur le même code.

Annexe

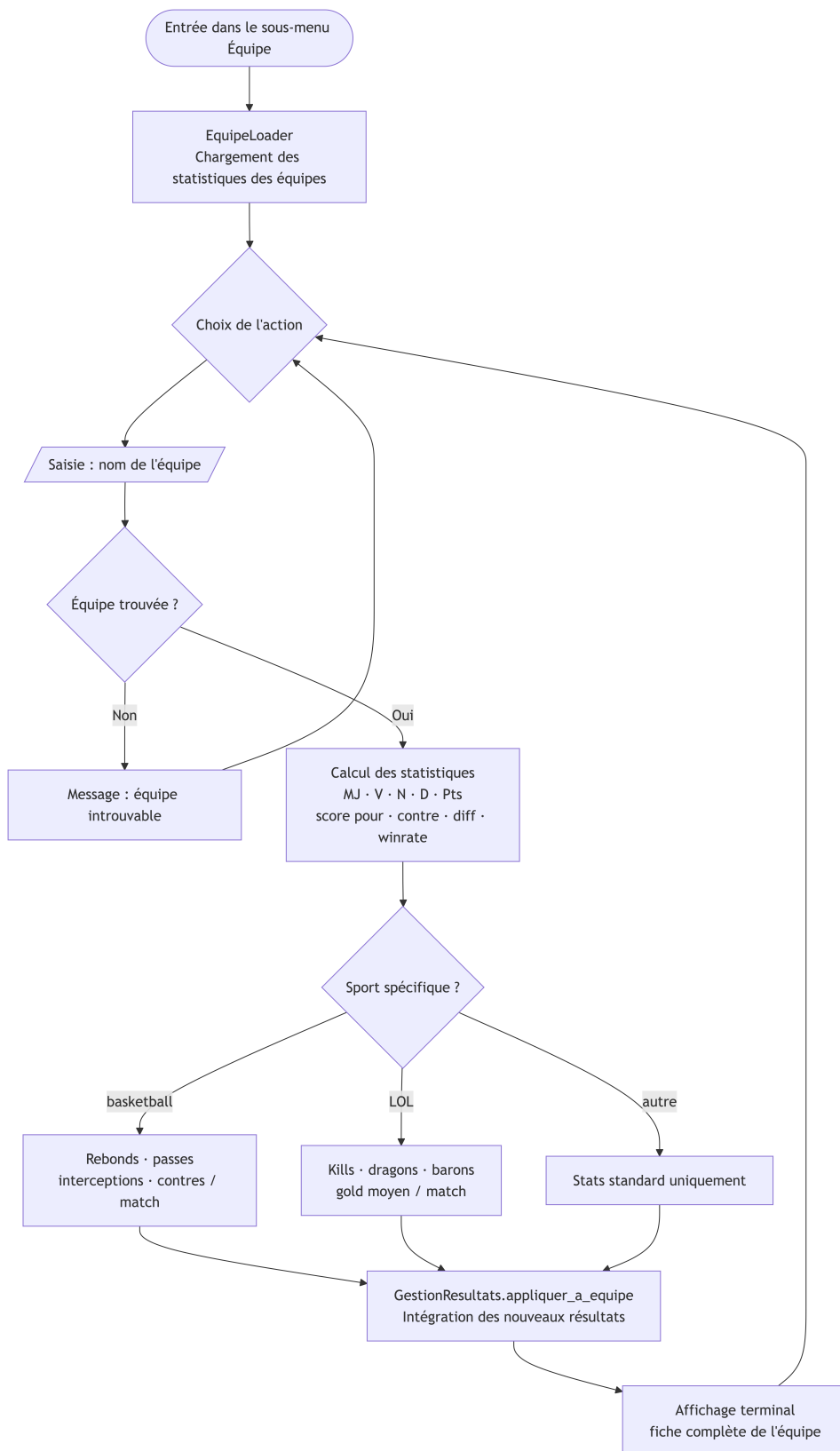
Diagrammes des sous-menus



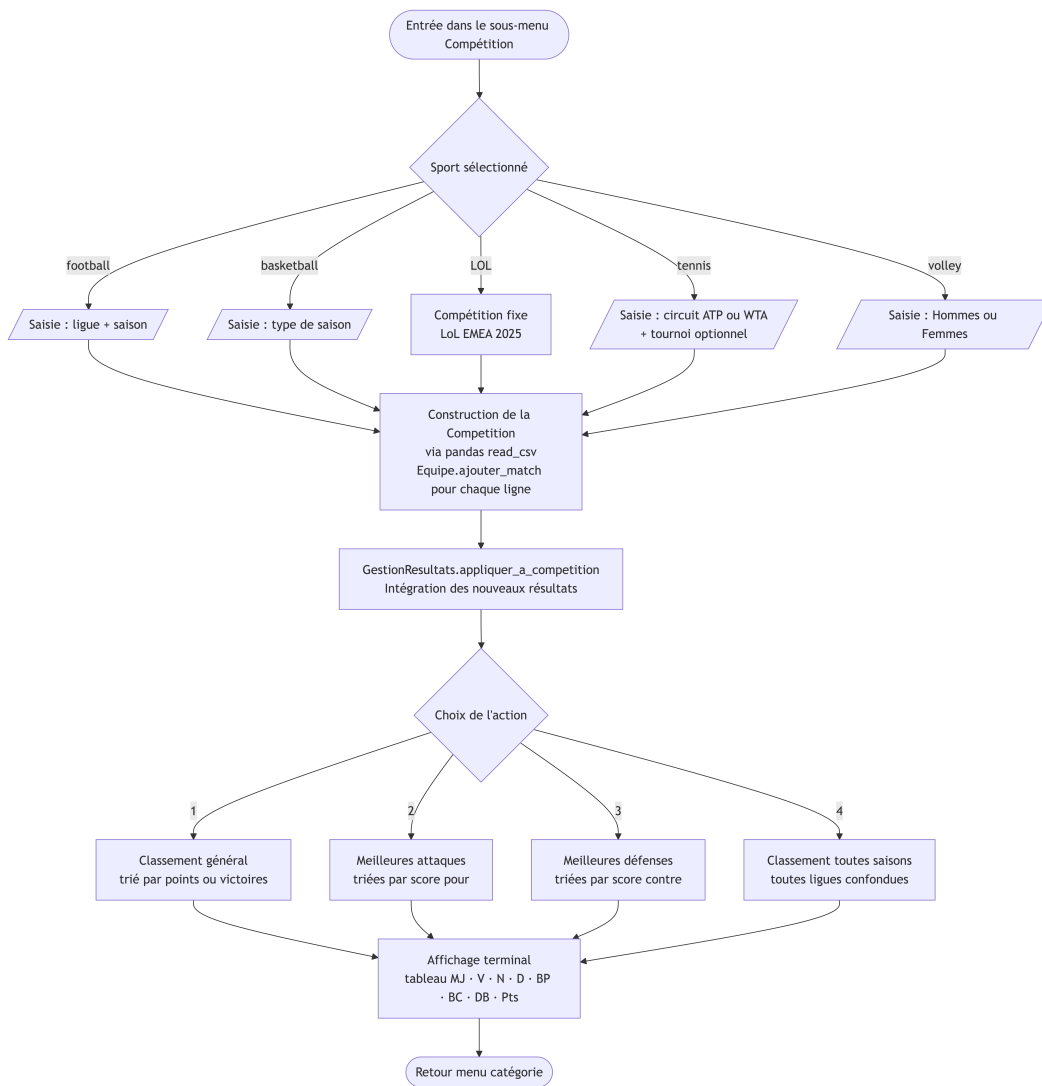
Sous-menu Joueur/Joueuse



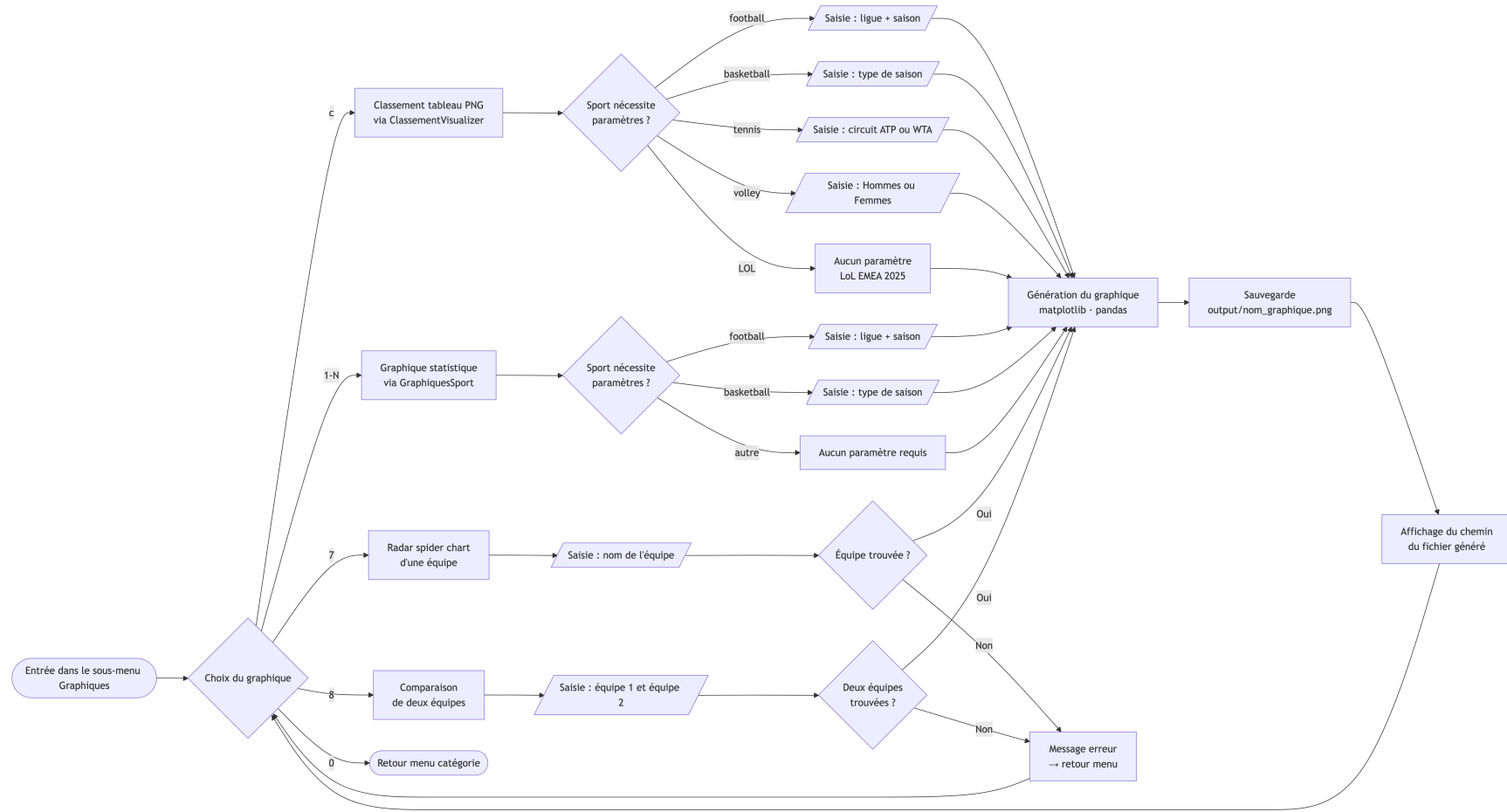
Sous-menu Match



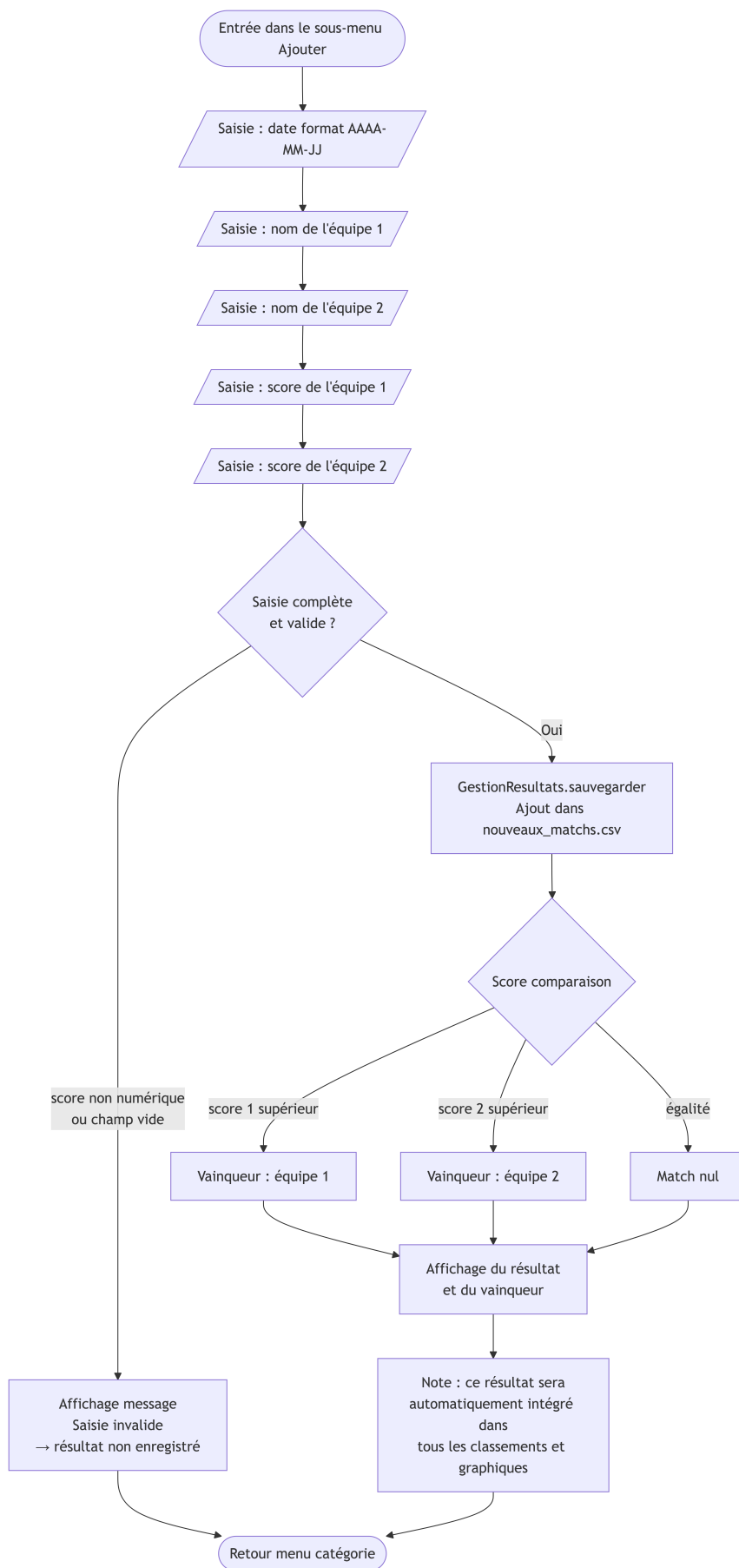
Sous-menu Équipe



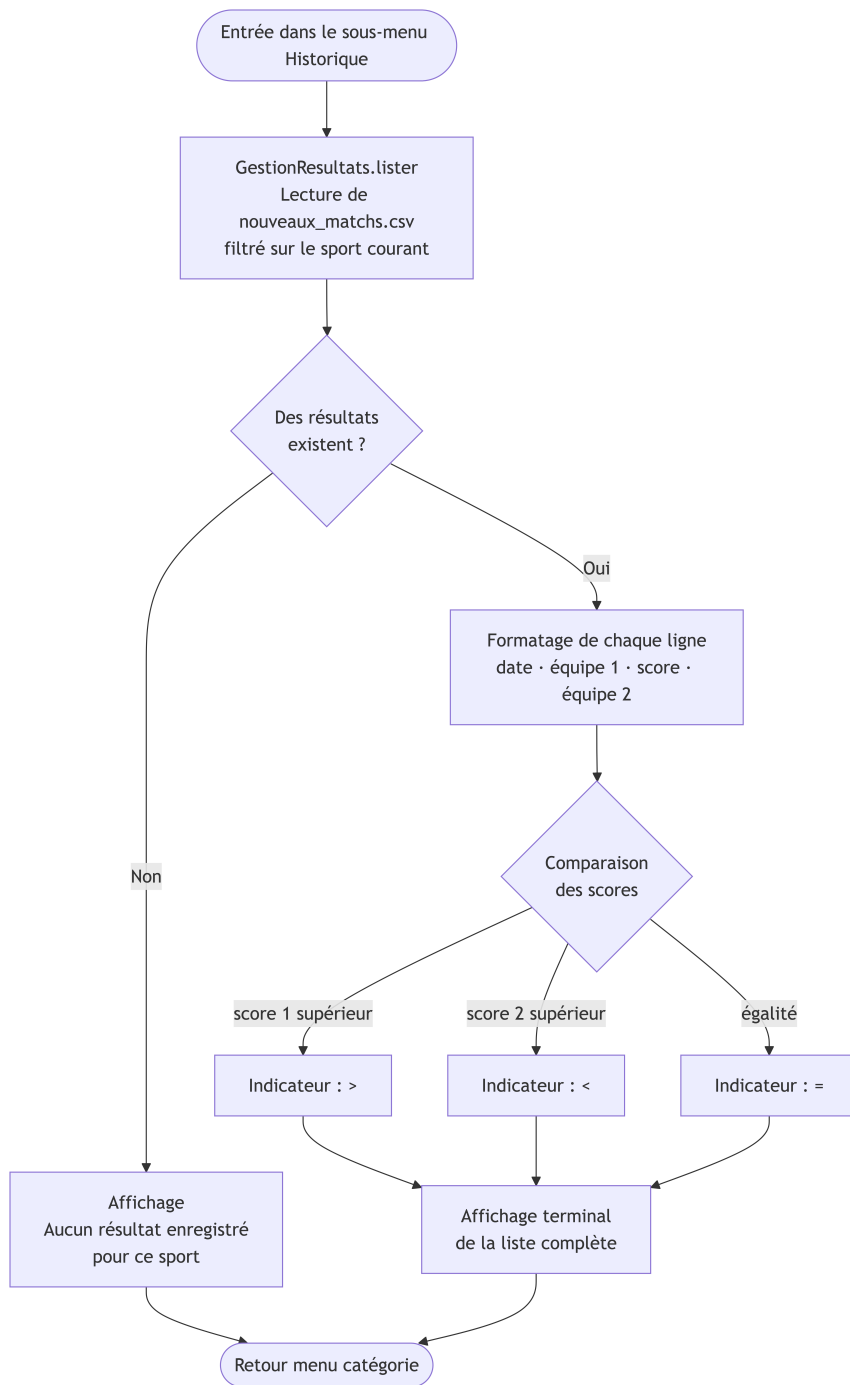
Sous-menu Compétition



Sous-menu Graphique



Sous-menu Ajouter



Sous-menu Historique